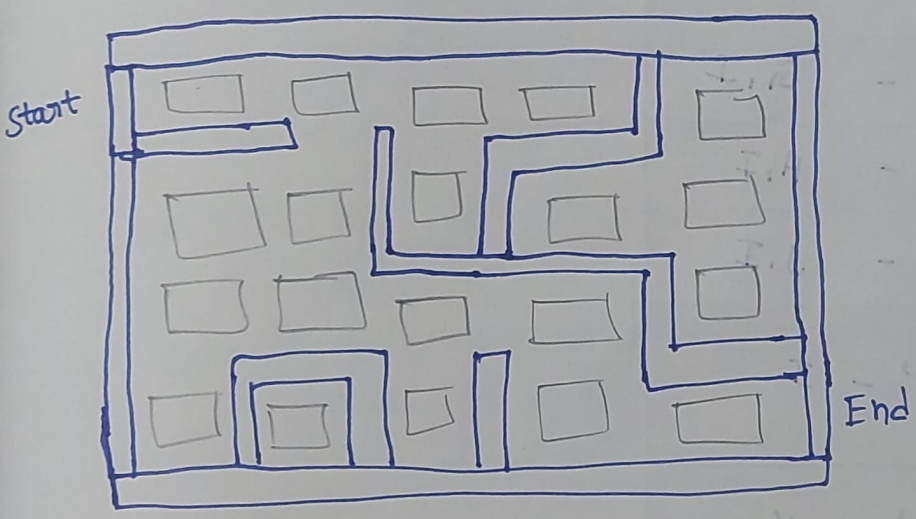
SOLVING MAZES using DFS

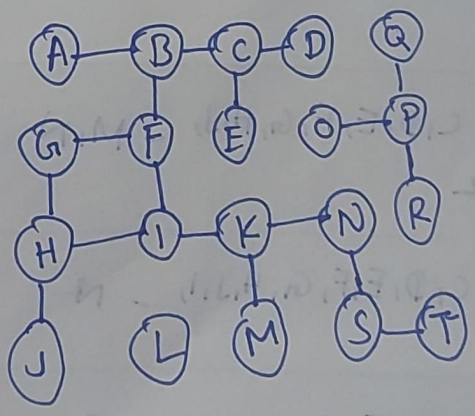
→ Divide / break down the maze into rows and columns, forming a grid-like structure. Each box forms a node / vertex of the graph.

→ Do a DFS of the formed graph, starting from the vertex which points to the start of the maze.

→ If, in the traversal, the vertex pointing to the end of the graph is reached, the maze has been solved.

Eg:



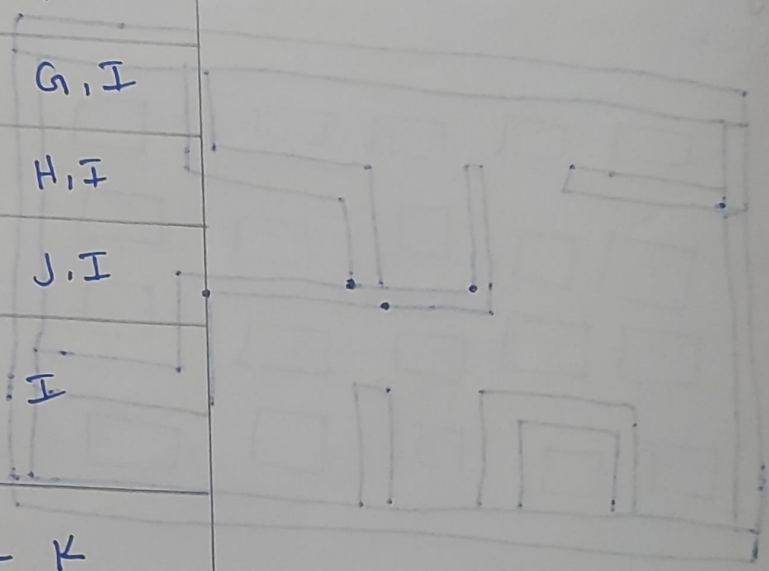


Start : A

End : T

→ We don't need to find the shortest path, we only need a solution, as our only motive is to solve the maze.

VISITED	-	STACK
A	-	B
A, B	-	C, F
A, B, C	-	D, E, F
A, B, C, D	-	E, F
A, B, C, D, E	-	F
A, B, C, D, E, F	-	G, I
A, B, C, D, E, F, G	-	H, I
A, B, C, D, E, F, G, H	-	J, I
A, B, C, D, E, F, G, H, J	-	I
A, B, C, D, E, F, G, H, J, I	-	K
A, B, C, D, E, F, G, H, J, I, K	-	M, N
A, B, C, D, E, F, G, H, J, I, K, M	-	N
A, B, C, D, E, F, G, H, J, I, K, M,	-	S
A, B, C, D, E, F, G, H, J, I, K, M, S	-	T



FINAL TRAVERSAL

A, B, C, D, E, F, G, H, J, I, K, M, S, T

STRONGLY CONNECTED COMPONENTS (SCCs)

1) KOSARAJU'S ALGORITHM

→ Perform DFS traversal of the graph to define the priorities of the vertices. Push the vertices onto a stack during traversal. Top of the stack has lowest ~~highest~~ priority.

→ Find the transpose graph (reverse edge directions).

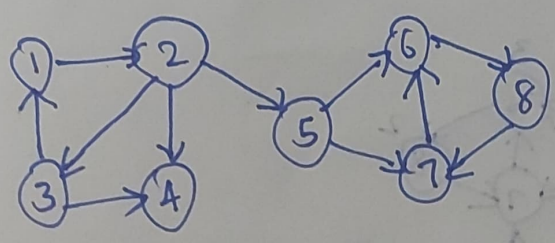
→ Perform DFS on the transpose graph. Start from the vertex with highest priority.

→ ~~Fro~~ Once an SCC is found, go to the next unvisited vertex with highest priority.

→ Once all vertices are visited, the algorithm is complete.

→ No. of DFS traversals of transpose graph = No. of SCCs.

Eg:

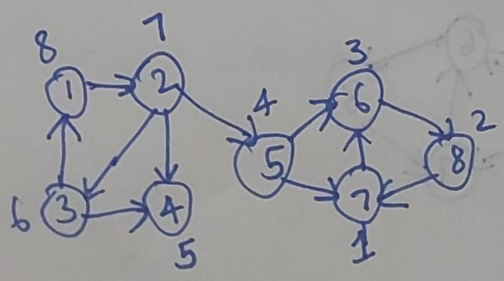


① DFS Traversal

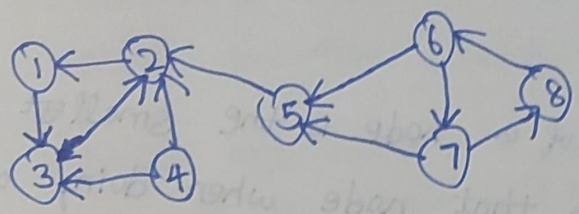
VISITED	—	STACK
1	—	2
1,2	—	3, 4, 5
1,2,3	—	4, 5
1,2,3,4	—	5
1,2,3,4,5	—	6, 7
1,2,3,4,5,6	—	8, 7
1,2,3,4,5,6,8	—	7
1,2,3,4,5,6,8,7		

DFS: 1, 2, 3, 4, 5, 6, 8, 7

Priority stack: 7, 8, 6, 5, 4, 3, 2, 1



② Transpose Graph



③ DFS of Transpose Graph based on Priority Stack

a) Highest Priority : 1
(unvisited)

Visited	- Stack
1	- 3
1,3	- 2
1,3,2	→ DFS

b) Highest Priority: 4
(unvisited)

DFS: 4 (2, 3 are already visited)

c) Highest Unvisited Priority: 5

DFS: 5

d) Highest Unvisited Priority: 6

Visited	- Stack
6	- 7
6,7	- 8
6,7,8	→ DFS

SCCs = 4

1) 1,3,2
2) 4
3) 5
4) 6,7,8

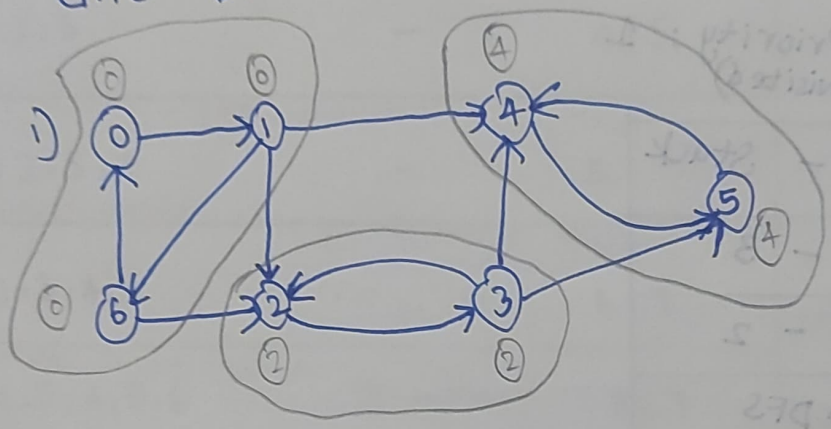
2) TARJAN'S ALGORITHM

LOW-LINK VALUES

→ Low-link value of a node is the smallest node ID reachable from that node when doing a DFS, including itself.

CATCH: DFS Traversal is random

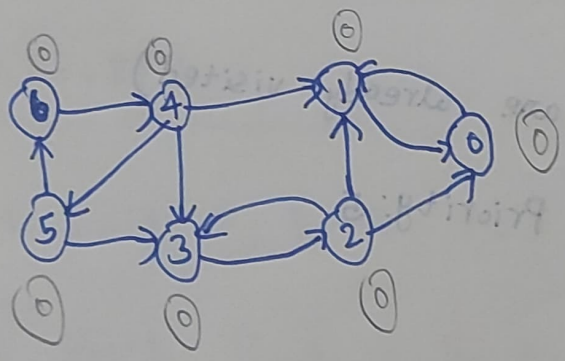
1)



3 SCCs

Labeling the same graph differently,

2)



It is the same graph, but we're getting only 1 SCC.

STACK INVARIANT

→ Maintain a stack of valid nodes from which to update low-link values.

→ Nodes are added to the stack of valid nodes as they are explored for the first time.

→ Nodes are removed from the stack each time a complete SCC is found.

If u and v are nodes and we are currently exploring u, the new-low-link condition update condition is:

→ To update node u to node v low-link, there has to be a path of edges from u to v, AND node v must be on the stack.

Let us redo Tarjan's algorithm with the stack invariant for the relabeled graph.

Current Vertex	Valid Node Stack	Low-Link Value
0	0	0
1	1, 0	0
Empty valid node stack as SCC is found. 0, 1 are marked as visited		
2	2	2
3	3, 2	2
Empty valid node stack.		

Current Vertex	Valid Node Stack	Low Link Value
4	4	4
5	5,4	4
6	6,5,4	4

Empty stack, since SCC has been found.

No. of SCCs = 3

Thus, with the updated low-link condition, Tarjan's algorithm is able to give the correct no. of SCCs regardless of the kind of DFS Traversal made.